Lab4-Flow Matching, Rectified Flow, and InstaFlow

Yu-Hsuan Li (112550011)
NYCU
johnnyli122022@gmail.com

1. Introduction

Diffusion models [4, 9] generate images by gradually removing noise, which usually takes tens to hundreds of network evaluations. Flow Matching (FM) [5] views generation as an ordinary differential equation (ODE): a network learns a velocity field $v_\theta(x, t)$ that moves samples from a Gaussian prior at $t = 0$ to the data distribution at $t = 1$, and sampling simply integrates this ODE. The training target is a simple regression onto the velocity of a straight path between a noise sample and a data sample.

The number of sampling steps is limited by how curved the learned trajectories are. Rectified Flow [6] straightens them by retraining on noise–image pairs produced by the model itself (*reflow*), and InstaFlow [7] goes one step further and distills the straightened flow into a *one-step* generator.

This assignment implements the full pipeline: (1) FM on a 2D Swiss-roll toy dataset, (2) class-conditional FM with classifier-free guidance (CFG) [3] on AFHQ [1] at 64×64, (3) 1-Rectified Flow for few-step sampling, and (4) (bonus) InstaFlow one-step distillation. All four tasks reach the full-score thresholds: Chamfer distance 13.49 (Task 1), FID 7.39 at CFG 7.5 (Task 2), FID 5.52 with 5 steps (Task 3) and FID 7.38 with a single step (Task 4).

2. Methodology

2.1. Flow Matching

Let $x_0 \sim \mathcal{N}(0, I)$ be noise and x_1 a data sample. FM defines a conditional path $\psi_t(x_0 \mid x_1)$ between them and trains v_θ to match the path's velocity $u_t = \frac{d}{dt}\psi_t$ (conditional FM loss, Eq. 23 of [5]):

$$\mathcal{L}_{\text{CFM}} = \mathbb{E}_{t,x_0,x_1}\left\|v_\theta(\psi_t, t) - u_t\right\|^2. \tag{1}$$

Although each conditional path is known, the loss has the same gradient as regressing onto the unknown marginal velocity field, so v_θ learns $\mathbb{E}[u_t \mid x_t]$. I use straight paths:

- **Task 1** (OT path): $\psi_t = (1 - (1 - \sigma_{\min})t)\, x_0 + t\, x_1$, $u_t = x_1 - (1 - \sigma_{\min})x_0$, with $\sigma_{\min} = 0.01$.
- **Tasks 2–4** (rectified path, as specified for the image tasks): $x_t = (1 - t)x_0 + t\, x_1$, $u_t = x_1 - x_0$.

t is drawn uniformly from $\{0, \frac{1}{1000}, \ldots, \frac{999}{1000}\}$ and x_0, x_1 are paired at random. The image loss (`image_common/fm.py`) is a direct translation of Eq. (1):

```
def get_loss(self, x1, class_label=None, x0=None):
    ... # t ~ U, x0 ~ N(0, I) (provided)
    t_ = t.view(-1, *([1] * (x1.dim() - 1)))
    x_t = (1 - t_) * x0 + t_ * x1 # point on the straight
        path
    u_t = x1 - x0 # constant target velocity
    model_out = self.network(x_t, t, class_label=
        class_label)
    return (model_out - u_t).pow(2).mean()
```

Task 1 is identical except for the OT path in `compute_psi_t` and the target u_t:

```
psi_t = (1 - (1 - self.sigma_min) * t) * x + t * x1 #
    compute_psi_t
ut = x1 - (1 - self.fm_scheduler.sigma_min) * x0 #
    get_loss
```

Sampling. Starting from $x_0 \sim \mathcal{N}(0, I)$, the ODE $\dot{x} = v_\theta(x, t)$ is integrated with N uniform Euler steps, $x_{t+\Delta t} = x_t + \Delta t\, v_t$, ending at $t = 1$.

Classifier-free guidance. For class-conditional generation the UNet receives a class label $c \in \{1, 2, 3\}$ (cat, dog, wild), whose embedding is added to the time embedding. During training the label is replaced by a null label 0 with probability 0.1, so a single network learns both $v(x, t \mid c)$ and $v(x, t \mid \varnothing)$. At sampling time the two are combined as

$$v_t = v(x_t, t \mid \varnothing) + w\left(v(x_t, t \mid c) - v(x_t, t \mid \varnothing)\right), \tag{2}$$

where $w > 1$ pushes samples further toward the class. The sampling loop combines the Euler step and CFG:

```
for i in range(num_inference_timesteps): # t: 0 -> 1
    ...
    dt = expand_t(t_next - t, xt)
    if do_classifier_free_guidance:
        null_labels = torch.zeros_like(class_label)
        v_cond = self.network(xt, t, class_label=
            class_label)
        v_uncond = self.network(xt, t, class_label=
            null_labels)
        vt = v_uncond + guidance_scale * (v_cond -
            v_uncond)
```

```
    else:
        vt = self.network(xt, t, class_label=class_label)
    xt = self.fm_scheduler.step(xt, vt, dt) # xt + dt * vt
```

Task 1 network. The velocity network reuses the Lab 1 `SimpleNet`: three time-conditioned linear layers (each output is scaled by an embedding of t) with ReLU, followed by a linear head. Lab 1 fed integer timesteps to the sinusoidal embedding, whereas FM uses $t \in [0, 1]$; I multiply t by 1000 before embedding so the embedding varies over its intended range.

```
def forward(self, x, t):
    t = t * self.num_timesteps # [0, 1] -> [0, 1000)
    for layer in self.layers: # TimeLinear x3
        x = F.relu(layer(x, t))
    return self.out_layer(x)
```

2.2. Rectified Flow (Task 3)

Because x_0 and x_1 are paired at random, straight conditional paths cross each other, and the averaged field $\mathbb{E}[u_t \mid x_t]$ produces *curved* trajectories that need many Euler steps. Reflow [6] replaces the random pairing with the deterministic pairing defined by the trained model:

1. sample $x_0 \sim \mathcal{N}(0, I)$ and a label c, and integrate the Task 2 model (20 steps, CFG 7.5) to obtain $z_1 = \Phi(x_0)$;
2. store $N = 50{,}000$ triplets (x_0, z_1, c);
3. train a new model from scratch with the same loss (Eq. (1)) on $x_t = (1-t)x_0 + tz_1$, $u_t = z_1 - x_0$.

ODE trajectories never cross, so the new pairs conflict far less and the retrained (1-RF) field is much straighter while generating the same distribution. The only implementation detail that matters is to start the ODE from the *saved* x_0 (passed explicitly to the sampler); otherwise the pairing is random again.

```
# generate_reflow_data.py (x_0 and labels sampled above)
z_1 = fm.sample(
    shape,
    num_inference_timesteps=args.num_inference_steps,
    class_label=labels,
    guidance_scale=args.cfg_scale if args.use_cfg else
        1.0,
    x0=x_0, # keep the (x_0, z_1) coupling
)
```

2.3. InstaFlow (Task 4)

If trajectories were perfectly straight, one Euler step $x_1 = x_0 + v(x_0, 0)$ would be exact. InstaFlow trains a student for exactly this one-step map (Eq. 6 of [7]):

$$\mathcal{L} = \left\| x_0 + v_\theta(x_0, 0 \mid c) - x_1 \right\|^2 + \mathcal{L}_{\text{LPIPS}}, \qquad (3)$$

where (x_0, x_1) are 50k pairs generated by the 1-RF teacher (20 steps, CFG $\alpha_2 = 1.5$) and LPIPS [11] is an optional perceptual loss. Guidance is baked into the targets, so sampling needs a single network evaluation without CFG.

Table 1. **Training configuration.** Task 1 uses a 3-layer MLP of width 128; Tasks 2–4 share the same UNet (channels 128, multipliers $[1, 2, 2, 2]$, attention at the 2nd level, 4 res-blocks per level). All runs use Adam on a single RTX 4090.

	Task 1	Task 2 / 3	Task 4
Network	MLP	UNet	UNet
Init.	random	random	1-RF
Batch	128	16	16
Steps	5k	100k	100k
LR	5×10^{-4}	2×10^{-4}	2×10^{-4}
Warm-up	–	200	5k + cosine
CFG dropout	–	0.1	0
Loss	CFM	CFM	L2 + LPIPS

```
# get_loss
t_zero = torch.zeros(x0.shape[0], device=self.device)
v_pred = self.network(x0, t_zero, class_label=
    class_label)
x1_pred = x0 + v_pred # one Euler step
loss = (x1_pred - x1).pow(2).mean()
if self.use_lpips:
    loss = loss + self.lpips_fn(x1_pred.clamp(-1, 1), x1.
        clamp(-1, 1)).mean()

# sample: one network call, no CFG
x_1 = x_T + self.network(x_T, t_zero, class_label=
    class_label)
```

Following the InstaFlow paper, I initialize the student from the 1-RF teacher instead of from scratch (Sec. 6). The full pipeline is: FM (CFG 7.5) $\rightarrow$ 1-RF $\rightarrow$ InstaFlow (CFG 1.5) $\rightarrow$ one step.

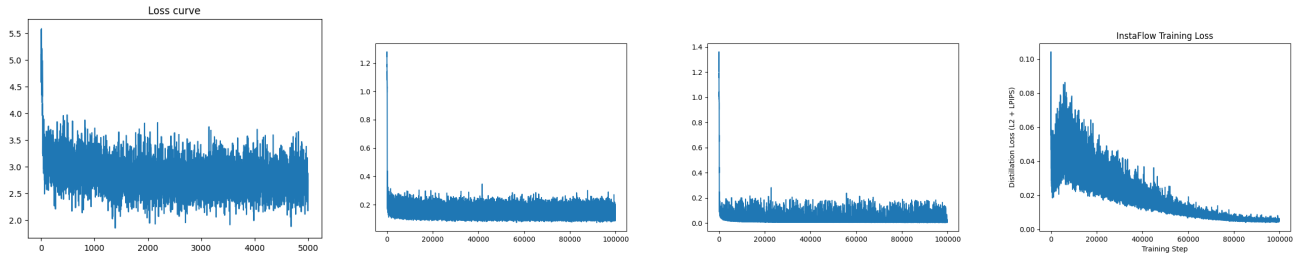
2.4. Implementation Details

Table 1 lists the configurations, which follow the provided defaults. Image quality is measured by FID [2] between 500 generated samples (random classes) and the 1,500 AFHQ validation images with the provided AFHQ Inception network. I used PyTorch 2.8, so checkpoints, which store the whole network object, were loaded with `TORCH_FORCE_NO_WEIGHTS_ONLY_LOAD=1`.

3. Experiments and Results

3.1. Task 1: 2D Flow Matching

The notebook trains for 5k iterations and evaluates the Chamfer distance (CD) between 2,048 generated and 2,048 target points with 50 Euler steps, giving **CD = 13.49** (< 40, full score). Figure 2 visualizes the conditional path and Fig. 3 the learned flow. The trajectories are nearly straight near the prior but bend near the end, where points must choose an arm of the spiral; this curvature is what Rectified Flow later reduces. Only a few points fall between the arms, which is the main source of the remaining CD.



(a) Task 1 (2D FM) (b) Task 2 (image FM) (c) Task 3 (1-RF) (d) Task 4 (InstaFlow)

Figure 1. **Training loss curves** (x-axis: training step). The FM losses plateau at a non-zero value because a point x_t is reachable from many random (x_0, x_1) pairs; with the reflow pairs of Task 3 this ambiguity mostly disappears and the loss is about $10\times$ lower. The bump in Task 4 comes from the 5k-step learning-rate warm-up.

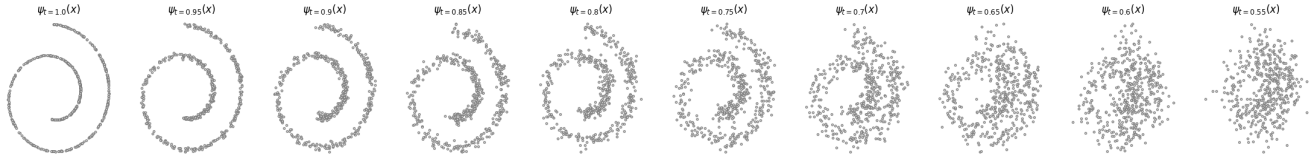


Figure 2. **Task 1: conditional path** $\psi_t(x_0 \mid x_1)$ applied to Swiss-roll samples, from $t = 1$ (data) to $t = 0.55$. Points move linearly toward the Gaussian prior.

Table 2. **Task 2: FID** (↓) of the base FM model, 500 samples. NFE: network evaluations per sample (CFG doubles it).

Setting	Steps	NFE	FID
CFG 1.0	20	20	12.18
CFG 3.0	20	40	12.13
CFG 7.5	**20**	**40**	**7.39**
CFG 7.5	10	20	8.14
CFG 7.5	50	100	5.47

Table 3. **Task 3: Base FM vs. 1-Rectified Flow** at CFG 7.5. FID (↓) on 500 samples; time is wall-clock for the 500 samples. Both models share the architecture, so the time per step is identical.

	FID		Time (s)	
Steps	Base FM	1-RF	Base FM	1-RF
5	8.72	**5.52**	14.4	14.2
10	9.16	**3.35**	28.5	28.5
20	5.84	**3.29**	56.9	57.0

3.2. Task 2: Image Flow Matching with CFG

Table 2 reports FID for different CFG scales and step counts, and Fig. 4 shows samples. **CFG 7.5 with 20 steps reaches FID 7.39** (< 30, full score).

- **CFG scale.** Larger guidance lowers FID (12.18 → 7.39). Visually, CFG 1.0 produces diverse but blurrier animals, while CFG 7.5 produces sharp, high-contrast faces that are less diverse (many black-and-white cats and dogs, saturated backgrounds).
- **Steps.** More steps help (10: 8.14, 20: 7.39, 50: 5.47), and the cost grows linearly with the number of network evaluations. The small gap between 10 and 20 steps already suggests that the learned trajectories are fairly straight.

3.3. Task 3: Rectified Flow

FID and speed. Table 3 compares the two models at the same step counts. **1-RF reaches FID 5.52 with only 5 steps** (< 30, full score), about the same as base FM with 20 steps (5.84). Since both models share the architecture, the time per step is identical; the speed-up comes from needing fewer steps. For similar quality (Fig. 6), 1-RF with 5 steps takes 14.2 s versus 56.9 s for base FM with 20 steps, a $\sim$**4**$\times$ **speed-up**. 1-RF also barely improves from 10 to 20 steps (3.35 → 3.29), so additional steps have little left to correct. Note that the reflow targets were generated with CFG 7.5 and CFG 7.5 is applied again at evaluation, and 500-sample FIDs are noisy (*e.g.* base FM at 5 vs. 10 steps), so small FID differences should not be over-interpreted.

Table 4. **Trajectory straightness** S (↓, Eq. (4)) over 256 trajectories of 20 Euler steps, using the same noise and labels for both models.

Sampling guidance	Base FM	1-RF
CFG 1.0 (learned field)	0.0222	**0.0016**
CFG 7.5	**0.0419**	0.5946

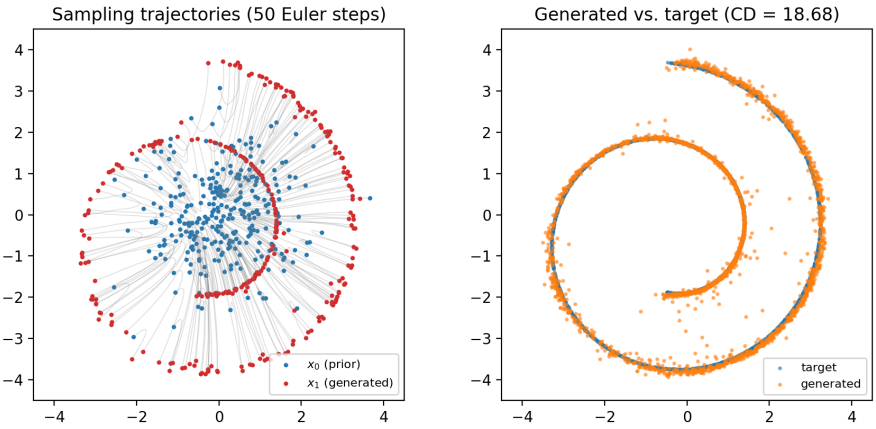


Figure 3. **Task 1: learned trajectories.** Left: 300 sampling trajectories from the prior (blue) to the generated points (red). Right: 2,048 generated points against the target. This figure is from a re-run with the same configuration (CD 18.68); the graded notebook run reached CD 13.49.



(a) CFG scale (20 steps)

(b) Number of steps (CFG 7.5)

Figure 4. **Task 2 samples** of the base FM model. Samples in different rows come from different noise.

Trajectory straightness. Following [6], I measure how far the velocity deviates from the overall displacement along a trajectory,

$$S = \mathbb{E}_t \left\| (x_1 - x_0) - v_t \right\|^2, \tag{4}$$

averaged per pixel ($S = 0$ for a straight line). Without guidance, 1-RF is about **14× straighter** than base FM (Tab. 4), which Fig. 5b shows visually. With CFG 7.5, however, 1-RF has a large S: almost all of the deviation is in the *first* step, where the conditional and unconditional velocities differ strongly and CFG amplifies the gap by 7.5×; the remaining steps are straight. This matters for Task 4 (Sec. 4).

Sampling without guidance. Since the reflow targets were generated with CFG 7.5, I also evaluate both models at CFG 1.0 (Tab. 5). Here the gap is much larger: **1-RF reaches FID 17.67 with one step and 10.14 with two**, while base FM needs 20 steps for 11.05. At 20 steps, 1-RF without guidance (6.96) matches its teacher (Task 2, CFG 7.5: 7.39), confirming that the guidance is baked into the 1-RF weights; adding CFG 7.5 on top still lowers FID for ≥5 steps. With 1–2 steps, however, CFG 7.5 breaks 1-RF (Fig. 7): the amplified first-step jump pushes pixels out of range. These broken samples still get a low FID, showing that 500-sample FID is unreliable for such failure modes.

Table 5. **Task 3: FID with and without guidance** (500 samples). †Most samples are saturated or blurry (Fig. 7); FID fails to reflect this, so these entries are not meaningful.

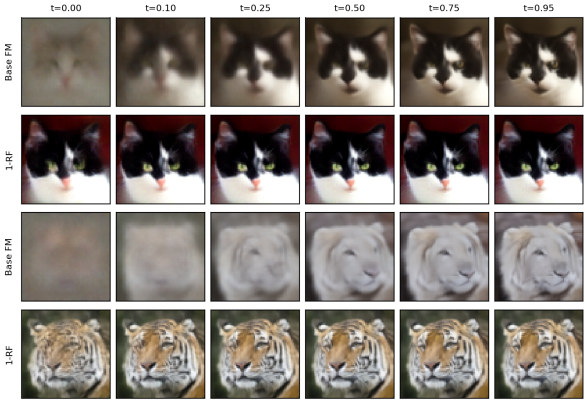
	CFG 1.0		CFG 7.5	
Steps	Base FM	1-RF	Base FM	1-RF
1	94.64	**17.67**	46.28	8.56†
2	80.88	**10.14**	9.07†	10.57†
5	36.56	**8.07**	8.72	**5.52**
10	17.06	**7.26**	9.16	**3.35**
20	11.05	**6.96**	5.84	**3.29**

3.4. Task 4 (Bonus): InstaFlow

Table 6 summarizes all stages. **InstaFlow reaches FID 7.38 with a single step** (< 30, full bonus), while generating 284 images/s: **99.6× faster** than base FM with 50 steps and **39.9× faster** than its 1-RF teacher. The cost is a mod-



(a) Samples (CFG 7.5) with 5 and 20 steps.



(b) Predicted endpoint along a trajectory (CFG 1.0).

Figure 5. **Task 3: base FM vs. 1-RF.** (b) shows the predicted endpoint $\hat{x}_1 = x_t + (1-t)\,v_t$ for the same noise: base FM starts from a blurry average and commits to details only gradually, while 1-RF already knows the final image at $t = 0$, *i.e.* its trajectory is almost a straight line.

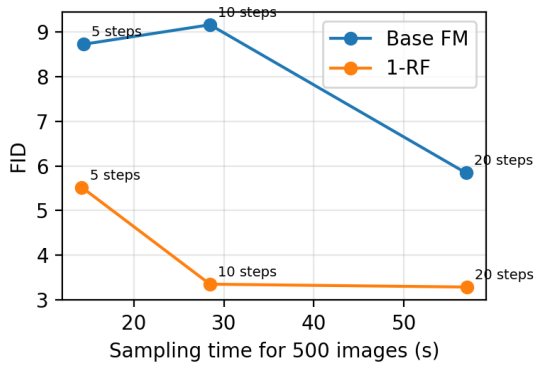


Figure 6. **Task 3: quality vs. sampling time** (CFG 7.5, 500 images). 1-RF reaches with 5 steps the FID that base FM needs 20 steps for.

Table 6. **Task 4: full pipeline.** FID (↓) on 500 samples. Timing for 500 samples was measured in one process on the same GPU (batch 128). Speed-up is relative to base FM (50 steps) and to the 1-RF teacher (20 steps).

Model	Steps	NFE	FID	Img/s	vs. FM	vs. 1-RF
Base FM (CFG 7.5)	50	100	5.47	2.9	1×	–
1-RF (CFG 1.5)	20	40	5.54	7.1	2.5×	1×
InstaFlow	**1**	**1**	**7.38**	**284**	**99.6×**	**39.9×**

erate quality drop (FID 5.47/5.54 → 7.38). Figure 8 shows no mode collapse: the one-step samples cover cats, dogs and wild animals with varied colors, but they are slightly blurrier than the multi-step samples. Because the student is initialized from the 1-RF teacher, it already produces recognizable animals in one step before any distillation update (initial loss 0.08, final 0.005; Fig. 9), so training only has to refine an already reasonable map.

Table 7. **Task 4 ablation.** Each variant changes one setting of the full model and is otherwise identical (same distillation data, 100k steps, seed). Loss is averaged over 500 steps (10 at step 0); it includes the LPIPS term except in the L2-only row, so that row is not comparable. FID (↓) on 500 one-step samples.

	Training loss at step			
Variant	0	20k	100k	FID
Full (L2 + LPIPS, 1-RF init)	0.080	0.031	0.005	7.38
L2 only	0.020	0.006	0.001	6.92
Random init	2.463	0.071	0.009	7.31

Ablation. I retrained the student twice, each time changing one setting (Tab. 7).

- **Without LPIPS** (L2 only), FID is 6.92 vs. 7.38, within the noise of a 500-sample FID, and the samples are visually indistinguishable (Fig. 10a).
- **Without 1-RF initialization**, the initial loss is 30× higher and the student needs roughly 20k steps to reach the loss the initialized student starts with (Fig. 10b), but after 100k steps it reaches the same FID (7.31).

All three students produce nearly the same image for the same noise, which suggests that the one-step map is mostly determined by the teacher's pairs, not by the loss or the initialization.

4. Discussion

4.1. Why does Rectified Flow enable faster sampling?

Euler integration is exact only for straight trajectories; on a curved trajectory a large step overshoots the curve, so few-



Figure 7. **Task 3: 1–2 step sampling with and without CFG.** With CFG 7.5, the single large first step of 1-RF overshoots and most images saturate; without guidance 1-RF already produces clean animals in one or two steps. Base FM with one step only produces a blurry average.



Figure 8. **Task 4 samples** from each stage of the pipeline: base FM (100 NFE), 1-RF teacher (40 NFE) and the one-step InstaFlow student (1 NFE).

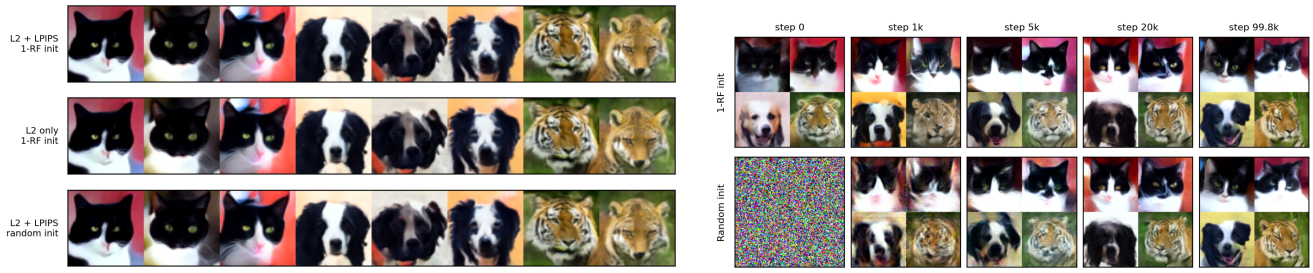
step sampling loses quality. Base FM is curved because it averages the directions of many randomly paired straight paths that cross at the same x_t. Reflow trains on pairs that are already connected by the model's own (non-crossing) ODE, so each x_t has essentially one target direction. This shows up in three ways in our results: a $10\times$ lower training loss (Fig. 1), a $14\times$ smaller straightness S (Tab. 4), and the endpoint already being predictable at $t = 0$ (Fig. 5b). As a result, 5 steps of 1-RF match 20 steps of base FM, and without guidance even 2 steps of 1-RF beat 20 steps of base FM (Tab. 5). 1-RF learns the teacher's distribution rather than the data: without extra guidance it reproduces the teacher's quality (FID 6.96 vs. 7.39), but in far fewer steps.

4.2. InstaFlow discussion questions

Q1: Why two phases instead of distilling FM directly? A one-step student must learn the map $x_0 \mapsto x_1$ in one shot. For base FM this map follows curved trajectories and is highly non-linear, so it is hard to fit and tends to give blurry averages. After reflow the trajectories are nearly straight, so the map is close to $x_0 + v(x_0, 0)$, which the 1-RF network



Figure 9. **Task 4: one-step samples during distillation** (fixed labels cat, cat, dog, wild). Thanks to the 1-RF initialization, the student generates plausible animals at step 0; distillation mainly sharpens details and removes artifacts.



(a) Final one-step samples from the same noise and labels.

(b) Samples during training, with and without 1-RF init.

Figure 10. **Task 4 ablation.** (a) All three students converge to almost the same one-step map. (b) The randomly initialized student starts from noise and is blurry at 1k steps, but catches up with the 1-RF-initialized one by about 5–20k steps.

already computes; distillation only corrects a small residual. This is also why initializing the student from 1-RF works so well.

Q2: Why not use LPIPS in Phase 1? Reflow regresses a velocity field at every t, including intermediate, noisy x_t. The L2 regression is what makes the learned field the conditional expectation of the velocity, which preserves the generated distribution; a perceptual loss is defined on clean images, is meaningless on velocities, and would bias the field away from a valid flow. In Phase 2 the student outputs a final image, so a perceptual loss applies naturally.

Q3: Effect of LPIPS in Phase 2. Conceptually, L2 alone penalizes pixel errors uniformly, and when the target is uncertain the optimum is a blurry average; LPIPS compares deep features and rewards sharp textures and edges. In our setting, however, LPIPS has no measurable effect: the L2-only student reaches a similar FID (6.92 vs. 7.38, Tab. 7) and visually the same samples (Fig. 10a). A likely reason is that the targets here are *not* uncertain: each noise has one deterministic teacher image, and the 1-RF student starts close to it, so L2 alone does not blur. LPIPS should matter more when the map is harder to fit, *e.g.* higher resolutions or a less straight teacher.

Q4: Quality–speed trade-off. From Tab. 6, going from 100 network evaluations (base FM) to 40 (1-RF) costs almost no quality (FID 5.47 vs. 5.54), and going to a single evaluation gives a further $40\times$ speed-up for $\sim$1.8 FID. For this task the one-step model is a good trade-off; when the best quality matters, 1-RF with 10–20 steps is preferable.

Q5: Why $\alpha_1 = 7.5$ but $\alpha_2 = 1.5$? In Phase 1 a high guidance scale makes the reflow targets class-specific and sharp, which gives a good teacher. Its effect is then stored in the 1-RF weights: without any guidance, 1-RF already matches the teacher's FID (Tab. 5). Applying a large scale again makes the 1-RF trajectory bend sharply in its first step ($S = 0.59$ vs. 0.0016 without guidance, Tab. 4). Multi-step sampling can correct this bend, but a one-step map cannot: sampling 1-RF with one step at CFG 7.5 produces mostly saturated images (Fig. 7), while one step without guidance already gives clean animals. A one-step student needs a nearly straight teacher, so a mild $\alpha_2 = 1.5$ keeps the trajectories straight while still adding some guidance (teacher FID 5.54, between 6.96 at CFG 1.0 and 3.29 at CFG 7.5). I did not train a student with a large α_2, so this reasoning is not verified by a full ablation.

4.3. Challenges

- **Time input of the Task 1 network.** The Lab 1 sinusoidal embedding expects integer timesteps up to 1000, but FM uses $t \in [0, 1]$, so the embedding barely changed over

time. Rescaling t by 1000 fixed this.

- **Keeping the reflow coupling.** The reflow pairs are only meaningful if z_1 is generated from the *saved* x_0. Letting the sampler draw its own noise would silently return random pairs, so I pass x_0 to the sampler explicitly.
- **A counter-intuitive straightness result.** At CFG 7.5, 1-RF first appeared *less* straight than base FM. Measuring per-step deviations showed that almost all of it comes from the CFG-amplified first step, while the learned field itself is $14\times$ straighter (Tab. 4).
- **Noisy evaluation.** 500-sample FID fluctuates (*e.g.* base FM is better with 5 steps than with 10) and even scores broken samples well (Fig. 7), so I always checked FID together with sample images.

4.4. Comparison with related work

DDIM [9] reduces sampling steps by a deterministic sampler but still needs tens of steps. Progressive distillation [8] and consistency models [10] also reach few-step generation, by repeatedly halving the step count or by enforcing self-consistency along trajectories. InstaFlow is simpler: reflow first makes the trajectories straight, after which a plain regression onto the endpoints suffices. Our result, one step with FID 7.38 against 5.47 for 50 steps, mirrors the paper's finding that reflow is the key to making one-step distillation work.

5. Conclusion

I implemented Flow Matching, Rectified Flow and InstaFlow and reached all full-score thresholds: CD 13.49 on the Swiss roll, FID 7.39 on AFHQ with CFG 7.5, FID 5.52 with 5 steps after one reflow, and FID 7.38 with a single step after distillation. The main lesson is that sampling cost is governed by trajectory curvature: reflow makes the learned field about $14\times$ straighter, which gives a $\sim 4\times$ speed-up at equal quality, and the straight field can then be distilled into a generator that is about $100\times$ faster than the original model. Guidance interacts with this: strong CFG re-introduces a large first-step jump, which is why distillation uses a mild guidance scale. The ablation shows that the distilled map is determined mostly by the teacher: neither LPIPS nor the 1-RF initialization changes the final FID, although the initialization speeds up convergence considerably.

Future work. Natural next steps are a second reflow (2-RF) as the distillation teacher, larger evaluation sets for less noisy FID, and higher-order ODE solvers for the multi-step models.

6. Additional Modified Files

Besides the required TODO files, I modified one script, included in the submission:

`task4_instaflow/train_instaflow.py`.

- **Change.** Added an optional argument `--init_ckpt` that loads the weights of a Flow Matching checkpoint (the 1-RF teacher) into the student UNet before training. Without the flag, the script behaves exactly as provided.
- **Rationale.** The InstaFlow paper initializes the one-step student from the rectified-flow teacher. The provided script trains it from random weights, and the assignment warns that one-step distillation is sensitive and can collapse. Starting from the teacher is a cheap way to avoid both slow convergence and collapse.
- **Effect.** The 1-RF velocity at $t = 0$ already approximates the endpoint, so the student generates recognizable images before any distillation update (initial loss 0.08 vs. 2.46 from random weights). The ablation (Tab. 7) shows that this mainly *speeds up convergence*: the random-init student needs about 20k steps to catch up, while the final FID after 100k steps is the same (7.38 vs. 7.31). With a shorter training budget, the initialization would matter more.

All other changes are inside the TODO blocks. Helper scripts in `scripts/` (run pipelines and report figures) only call the provided entry points and do not change their behavior.

References

[1] Yunjey Choi, Youngjung Uh, Jaejun Yoo, and Jung-Woo Ha. StarGAN v2: Diverse image synthesis for multiple domains. In *IEEE Conf. Comput. Vis. Pattern Recog.*, 2020. 1

[2] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. GANs trained by a two time-scale update rule converge to a local Nash equilibrium. In *Adv. Neural Inform. Process. Syst.*, 2017. 2

[3] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. In *NeurIPS Workshop on Deep Generative Models and Downstream Applications*, 2021. 1

[4] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Adv. Neural Inform. Process. Syst.*, 2020. 1

[5] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *Int. Conf. Learn. Represent.*, 2023. 1

[6] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *Int. Conf. Learn. Represent.*, 2023. 1, 2, 4

[7] Xingchao Liu, Xiwen Zhang, Jianzhu Ma, Jian Peng, and Qiang Liu. InstaFlow: One step is enough for high-quality diffusion-based text-to-image generation. In *Int. Conf. Learn. Represent.*, 2024. 1, 2

[8] Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. In *Int. Conf. Learn. Represent.*, 2022. 8

[9] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *Int. Conf. Learn. Represent.*, 2021. 1, 8

[10] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In *Int. Conf. Mach. Learn.*, 2023. 8

[11] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *IEEE Conf. Comput. Vis. Pattern Recog.*, 2018. 2